

README - Project Overview

Setup, features, API reference and repository structure

FitBuddy — AI-Powered Personal Fitness Coach

A full-stack AI fitness application powered by FastAPI + Google Gemini AI

FastAPI

0.115.5

Python

3.9+

Gemini

2.0 Flash

SQLite

3

Project Overview

FitBuddy is an AI-powered fitness coaching platform that generates **personalized 7-day workout plans**, adapts them based on **natural language feedback**, and provides **nutrition and recovery tips** — all powered by Google's Gemini 2.0 Flash AI model.

Key Features

Feature	Description
 AI Workout Plans	Generate 7-day personalized plans based on age, weight, goal & intensity
 Smart Feedback	Update your plan with natural language like "More cardio, fewer rest days"
 Nutrition Tips	Get AI-generated dietary and recovery advice for your fitness goal

■ History Tracking	All plans and tips stored in SQLite for future reference
■ Premium UI	Glassmorphism dark mode with animations and responsive design

■ Quick Start

1. Get a Gemini API Key

Visit [Google AI Studio](#) and create a free API key.

2. Install Dependencies

```
cd backend pip install -r requirements.txt
```

3. Set Your API Key

```
# Windows PowerShell $env:GEMINI_API_KEY = "your-key-here"
```

```
# Linux / macOS export GEMINI_API_KEY="your-key-here"
```

4. Run the Server

```
cd backend uvicorn main:app --reload --port 8000
```

5. Open in Browser

Navigate to <http://localhost:8000>

■ **Windows users:** Double-click `start.bat` to launch automatically.

■■ Repository Structure

```
Fitbuddy/ ■■■ ■ 1_Ideation_Phase/ # Problem statement, brainstorming, empathy map ■■■ ■  
2_Requirement_Analysis/ # Functional & non-functional requirements ■■■ ■ 3_Frontend_Development/ #  
Frontend architecture & design docs ■■■ ■ 4_Backend_Development/ # Backend API & database  
documentation ■■■ ■ 5_AI_ML_Integration/ # Prompt engineering & AI integration docs ■■■ ■  
6_System_Testing/ # Unit, integration & performance tests ■■■ ■ 7_Deployment/ # Docker, Render,
```

```
CI/CD deployment guide 8_Project_Documentation/ # Complete project report backend/
main.py # FastAPI application (all routes & logic) requirements.txt # Python
dependencies frontend/ index.html # Single-page application static/
style.css # Complete design system (880 lines) app.js # Application logic (478 lines)
README.md # This file start.bat # Windows quick-start script
```

Technology Stack

Layer	Technology
Backend	FastAPI 0.115.5 + Uvicorn
AI Engine	Google Gemini 2.0 Flash
Database	SQLite3
Frontend	Vanilla HTML5 / CSS3 / JavaScript
Fonts	Outfit + Space Grotesk (Google Fonts)

API Endpoints

Method	Endpoint	Scenario	Description
POST	/api/generate-plan	Scenario 1	Generate 7-day personalized workout plan
POST	/api/update-plan	Scenario 2	Update plan with user feedback
POST	/api/nutrition-tip	Scenario 3	Get nutrition & recovery tips
GET	/api/user/{id}/history	—	View user plan history
GET	/api/health	—	Service health check

Interactive API Docs: Visit <http://localhost:8000/docs> when server is running.

■ Project Phases

This project was developed following a structured lifecycle:

Phase	Folder	Description
Ideation	1_Ideation_Phase/	Problem statement, brainstorming, empathy map
Requirements	2_Requirement_Analysis/	Functional, non-functional, API, DB requirements
Frontend	3_Frontend_Development/	UI components, CSS design system, JavaScript logic
Backend	4_Backend_Development/	FastAPI routes, Pydantic models, SQLite integration
AI/ML	5_AI_ML_Integration/	Prompt engineering, Gemini integration, parsing
Testing	6_System_Testing/	Unit, integration, performance, edge case tests
Deployment	7_Deployment/	Docker, cloud deploy, CI/CD pipeline
Documentation	8_Project_Documentation/	Complete project report

■ Sample API Usage

Generate a Workout Plan

```
curl -X POST http://localhost:8000/api/generate-plan \ -H "Content-Type: application/json" \ -d '{"name": "Alex", "age": 25, "weight": 70.0, "goal": "weight_loss", "intensity": "medium"}'
```

Update Plan with Feedback

```
curl -X POST http://localhost:8000/api/update-plan \ -H "Content-Type: application/json" \ -d '{"user_id": 1, "feedback": "I want more cardio and fewer rest days"}'
```

Get Nutrition Tips

```
curl -X POST http://localhost:8000/api/nutrition-tip \ -H "Content-Type: application/json" \ -d '{"user_id": 1, "goal": "weight_loss"}'
```

■ UI Screenshots

The FitBuddy interface features:

- **Dark glassmorphism cards** with blur and transparency
- **Animated gradient background orbs** for visual depth
- **Interactive day cards** that expand to show full workout details
- **Real-time progress bar** during AI plan generation
- **Toast notifications** for success/error feedback
- **Animated hero statistics** counters on page load

■ Documentation

All project documentation is organized in the numbered phase folders:

- ■ [Problem Statement](#)
- ■ [Brainstorming](#)
- ■ [Empathy Map](#)
- ■ [Requirement Analysis](#)
- ■ [Frontend Documentation](#)
- ■ [Backend Documentation](#)
- ■ [AI/ML Integration](#)
- ■ [Testing Documentation](#)
- ■ [Deployment Guide](#)
- ■ [Project Report](#)

■ About

FitBuddy was built as a demonstration of generative AI applied to personal health and fitness.

Built with ♥■ using:

- [FastAPI](#) — Modern Python web framework
- [Google Gemini](#) — State-of-the-art language model
- [SQLite](#) — Lightweight embedded database

FitBuddy © 2025 — Powered by Google Gemini AI & FastAPI